CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 04/Unit 04: Memory Leaks, Function Overloading

Course Taught by: Zubair Irshad

Slide Courtesy: Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk)

Unit Outline

1. Memory Leaks
2. Function Overloading
3. Introduction to Object Oriented Programming
4. Summary

Memory Leaks (Overview)

How to delete allocated memory!

Memory Leak in Dynamic Memory Allocation

- This allocated dynamic memory must be explicitly deallocated after use, since it retains itself after the block of code that requested the allocation ends.
- To see this, have a look at the following program:

Memory Leak

```cpp
#include <iostream>

int* global;
//global variable to demonstrate memory leak

void funct(void) {
    int* arr = new int[10];
    std::cout << "Inside Function funct" << "\n";
    for (int i = 0; i < 10; ++i) {
        arr[i] = i * 2;
        std::cout << "arr[" << i << "] = " << arr[i] << ", ";
    }
    std::cout << "\n";
    global = arr;
    // Memory leak: 'delete[] arr;' is missing
    // Function ends without deallocating the allocated memory
}
```

```cpp
int main() {
    // Allocate an array of 10 integers
    funct();
    std::cout << "Inside Main " << "\n";

    for (int i = 0; i < 10; ++i) {
        std::cout << "arr[" << i << "] = " << *(global + i) << ", ";
    }
    std::cout << "\n";
    return 0;
}
```

- **What is the correct way to deallocate memory?**

Memory Leak

```cpp
#include <iostream>

int* global;
//global variable to demonstrate memory leak

void funct(void) {
    int* arr = new int[10];
    std::cout << "Inside Function funct" << "\n";
    for (int i = 0; i < 10; ++i) {
        arr[i] = i * 2;
        std::cout << "arr[" << i << "] = " << arr[i] << ", ";
    }
    std::cout << "\n";
    global = arr;
    // Memory leak: 'delete[] arr;' is missing
    // Function ends without deallocating the allocated memory
}
```

```cpp
int main() {
    // Allocate an array of 10 integers
    funct();
    std::cout << "Inside Main " << "\n";

    for (int i = 0; i < 10; ++i) {
        std::cout << "arr[" << i << "] = " << *(global + i) << ", ";
    }
    std::cout << "\n";
    return 0;
}
```

- **What is the correct way to deallocate memory?**
  - Always call **delete [] arr** when you initialize a dynamic array with **new** operator

Dangling Pointers

- A pointer pointing to a memory location that has explicitly been deleted is known as a dangling pointer.
  - Pointer value i.e. array is **freed** but the pointer still stores the location of it.
- A dangling pointer must be set to nullptr.
- In the last example, we must use

```
    delete[] arr; //to delete the allocated memory
    global = nullptr; //to set the pointer to null
```

Function Overloading

Function Overloading

- **Function overloading** is a programming concept that allows multiple functions to have the same name but differ in the **number** or **type of their parameters**.
- Why need it? It enables programmers to create multiple versions of a function tailored to different input types or use cases, while maintaining intuitive naming.
- A function is said to be **overloaded** if an implementation of it has the same name but different signature.
- The signature of a function is the **number**, **type** and **order of its parameters**.
- Note that you may get a compiler error, if a function name and signature is the same, however the return type is different.
- **Important!!** Return Type Alone Can't Be Used for Overloading

Function Overloading (Examples)

```cpp
#include <iostream>

// Function to print an integer
void printData(int value) {
    std::cout << "Integer: " << value << std::endl;
}

// Function to print a floating-point number
void printData(double value) {
    std::cout << "Double: " << value << std::endl;
}

// Function to print a character array (C-string)
void printData(const char* value) {
    std::cout << "Character Array: " << value << std::endl;
}
```

Function Overloading (Examples)

```
int main() {
    int intValue = 42;
    double doubleValue = 3.14;
    const char* charArrayValue = "Hello, World!";

    // Call the overloaded functions
    printData(intValue);          // Calls printData(int)
    printData(doubleValue);       // Calls printData(double)
    printData(charArrayValue);    // Calls printData(const char*)

    return 0;
}
```

Function Overloading (Allowed vs Not allowed!)

```
void print(int x);          // Signature: (int)
void print(double y);       // Signature: (double)
void print(int x, int y); // Signature: (int, int)
```

```
int calculate(int a);
double calculate(int a); // ❌ Error: Same name and signature, only return type differs
```

Function Overloading (Another Example!)

```cpp
// Overloaded functions
int add(int a, int b) {
    return a + b;
}

double add(double a, double b) {
    return a + b;
}

int add(int a, int b, int c) {
    return a + b + c;
}

int main() {
    cout << add(2, 3) << endl;         // Calls int version
    cout << add(2.5, 3.5) << endl;     // Calls double version
    cout << add(1, 2, 3) << endl;      // Calls 3-parameter version
}
```

Summary

- In C++, dynamically allocated memory must be explicitly deallocated in order to avoid memory leaks
- An associated concept to memory leaks is that of a dangling pointer, which is a pointer that refers to a variable or a data element that has been deleted and deallocated.
- Function overloading is the concept of assigning the same functionality to a function using different parameter types, order or number.
- Overloaded functions must have the same name.